

```

import numpy as np
import pandas as pd
import yfinance as yf
import matplotlib.pyplot as plt
from sklearn.preprocessing import MinMaxScaler
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense, Dropout

# Download Castrol India stock data for the last 1 month
stock_data = yf.download('CASTROLIND.NS', period='1mo', interval='1d')

# Display the first few rows of data
print(stock_data.head())

# Select only the 'Close' prices for prediction
close_prices = stock_data[['Close']].values

# Normalize the 'Close' prices
scaler = MinMaxScaler(feature_range=(0, 1))
scaled_close_prices = scaler.fit_transform(close_prices)

# Prepare the data for LSTM (sequence data)
time_step = 3 # Reduce the time_step to 3 for small datasets
X = []
y = []

# Create sequences of 3 days to predict the next day's 'Close' price
for i in range(len(scaled_close_prices) - time_step):
    X.append(scaled_close_prices[i:(i + time_step)])
    y.append(scaled_close_prices[i + time_step, 0]) # 'Close' is the 0th column

X = np.array(X)
y = np.array(y)

# Check the shape of X to confirm the data is reshaped correctly
print(f"Shape of X: {X.shape}")
print(f"Shape of y: {y.shape}")

# Reshape X to be compatible with LSTM input shape
X = X.reshape(X.shape[0], X.shape[1], 1)

# Split the data into training and testing sets
train_size = int(len(X) * 0.8)
X_train, X_test = X[:train_size], X[train_size:]
y_train, y_test = y[:train_size], y[train_size:]

# Build the LSTM model
model = Sequential()

# Add LSTM layers
model.add(LSTM(units=50, return_sequences=True, input_shape=(X_train.shape[1], 1)))
model.add(Dropout(0.2)) # Dropout to avoid overfitting
model.add(LSTM(units=50, return_sequences=False))
model.add(Dropout(0.2))

# Add the output layer
model.add(Dense(units=1)) # Predicting a single value (Close price)

# Compile the model
model.compile(optimizer='adam', loss='mean_squared_error')

# Train the model
history = model.fit(X_train, y_train, epochs=10, batch_size=32, validation_data=(X_test, y_test))

# Future Predictions (Next 7 days)

# Get the last 3 days of data for prediction
last_3_days = scaled_close_prices[-time_step:]

# Reshape the data to the model's expected input shape
current_input = last_3_days.reshape(1, time_step, 1)

# Predict the next 7 days
future_predictions = []
for _ in range(7): # Predict the next 7 days
    predicted_price = model.predict(current_input)

    # Append the predicted price
    future_predictions.append(predicted_price[0][0])

    # Update the input with the predicted price (reshape predicted_price to match the shape of current_input)
    current_input = np.append(current_input[:, 1:, :], predicted_price.reshape(1, 1, 1), axis=1)

# Reverse the scaling of the predicted prices
future_predictions_rescaled = scaler.inverse_transform(np.array(future_predictions).reshape(-1, 1))

```

```
# Generate future dates for the next 7 business days
future_dates = pd.date_range(start=stock_data.index[-1], periods=8, freq='B')[1:]

# Display the predicted prices along with dates
print("Predicted Stock Prices for the Next 7 Days:")
predicted_prices = list(zip(future_dates.strftime('%Y-%m-%d'), future_predictions_rescaled.flatten()))
for date, price in predicted_prices:
    print(f>Date: {date}, Predicted Price: ₹{price:.2f}")

# Plotting the actual stock prices and predicted future prices
plt.figure(figsize=(14, 6))

# Plot the actual stock prices (Close prices)
plt.plot(stock_data.index[-30:], stock_data['Close'][-30:], label='Actual Castrol India Stock Price', color='blue')

# Plot the predicted future stock prices
plt.plot(future_dates, future_predictions_rescaled, label='Predicted Castrol India Stock Price (Next 7 Days)', color='green')

# Title and labels
plt.title('Castrol India Stock Price - Prediction for Next 7 Days')
plt.xlabel('Date')
plt.ylabel('Stock Price')
plt.legend()
plt.xticks(rotation=45)
plt.grid(True)
plt.show()
```

Ticker	CASTROLIND.NS	CASTROLIND.NS	CASTROLIND.NS	CASTROLIND.NS
Date				
2025-02-21	210.856400	213.763482	208.096148	209.368596
2025-02-24	209.926529	210.709584	206.432151	209.417541
2025-02-25	212.716171	213.949473	210.004848	210.445312
2025-02-27	213.841797	217.199126	212.216960	215.339383
2025-02-28	208.125519	214.243115	206.148308	213.381753

Price	Volume
Ticker	CASTROLIND.NS
Date	
2025-02-21	6079188
2025-02-24	2951632
2025-02-25	4201668
2025-02-27	5889216
2025-02-28	7794409
Shape of X: (16, 3, 1)	
Shape of y: (16,)	
Epoch 1/10	

```
/usr/local/lib/python3.11/dist-packages/keras/src/layers/rnn/rnn.py:200: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a 1
super().__init__(**kwargs)
1/1 ----- 2s 2s/step - loss: 0.3716 - val_loss: 0.2163
Epoch 2/10
1/1 ----- 0s 74ms/step - loss: 0.3469 - val_loss: 0.2015
Epoch 3/10
1/1 ----- 0s 70ms/step - loss: 0.3373 - val_loss: 0.1872
Epoch 4/10
1/1 ----- 0s 69ms/step - loss: 0.3205 - val_loss: 0.1733
Epoch 5/10
1/1 ----- 0s 75ms/step - loss: 0.3073 - val_loss: 0.1598
Epoch 6/10
1/1 ----- 0s 69ms/step - loss: 0.2829 - val_loss: 0.1466
Epoch 7/10
1/1 ----- 0s 68ms/step - loss: 0.2606 - val_loss: 0.1337
Epoch 8/10
1/1 ----- 0s 69ms/step - loss: 0.2406 - val_loss: 0.1213
Epoch 9/10
1/1 ----- 0s 79ms/step - loss: 0.2298 - val_loss: 0.1094
Epoch 10/10
1/1 ----- 0s 84ms/step - loss: 0.2120 - val_loss: 0.0982
1/1 ----- 0s 164ms/step
1/1 ----- 0s 31ms/step
1/1 ----- 0s 29ms/step
1/1 ----- 0s 27ms/step
1/1 ----- 0s 27ms/step
1/1 ----- 0s 31ms/step
1/1 ----- 0s 29ms/step
Predicted Stock Prices for the Next 7 Days:
Date: 2025-03-24, Predicted Price: ₹213.13
Date: 2025-03-25, Predicted Price: ₹212.79
Date: 2025-03-26, Predicted Price: ₹212.56
Date: 2025-03-27, Predicted Price: ₹212.11
```